```
      create  aaagmnr/.gitignore
      create  aaagmnr/.rspec
      create  aaagmnr/spec/spec_helper.rb
      create  aaagmnr/spec/aaagmnr_spec.rb
      create  aaagmnr/.github/workflows/main.yml
      create  aaagmnr/LICENSE.txt
      create  aaagmnr/CODE_OF_CONDUCT.md
      create  aaagmnr/CHANGELOG.md
      create  aaagmnr/.standard.yml
Gem 'aaagmnr' was successfully created.
For more information on making a RubyGem visit
https://bundler.io/guides/creating_gem.html
```

The "config" referred to is the `bundle config`, which contains some global options for bundler, and which you can update with `bundle config set ci github` or whatever the config variable and setting you want. Anything in the configuration can also be set using command-line options to the `bundle gem` command itself.

What has Bundler created for us?

- A `aaagmnr.gemspec` file. The gemspec file contains all the metadata that RubyGems uses to manage this gem. We'll talk about that in a second in Distributing and Installing Your Code, on page 272. We've also got a regular Gemfile.
- A series of logistical files, including an open source license, a basic change log, a code of conduct file, a readme, and a Rakefile. Some of these are managed by the configuration, including the type of license and whether to include the code of conduct.
- The `lib` directory, which is where our code will go.
- The `sig` directory, which is where Ruby type info would go if we were using Ruby types. See Chapter 7, Standard Types, on page 117.
- A `bin` directory, with some useful pre-created setup scripts.
- We've been set up as a Git repo; you can choose other source control repositories in the config.
- We've been set up with a `spec` directory and RSpec; you can choose a different testing tool in the config.
- A linter file; we're using the Standard gem,[3] the choice is a configurable option.

That's quite a lot of work already done for us. Now we can fit our code inside that structure.

Let's look at the `lib` directory. This is where we put our source code. Right now it has two files in it, `aaagmnr.rb`, which will be our top level access to the code, and `aaagmnr/version.rb`, which contains a `VERSION` constant. We will see more of this later.

The intention is that our actual logic will go alongside that `version.rb` in the `lib/aaagnmr` directory. Why is that?

We know we're going to be defining (at least) three classes. Right now, these classes will be used only by our command-line program, but it's conceivable that other people might want to include one or more of our libraries in their own code. This means that we should be polite and not pollute the top-level Ruby namespace with the names of all our classes and so on. We'll create just one top-level module, `Aaagnmr`, and then place all our classes inside this module. This means that the full name of (say) our options-parsing class will be `Aaagnmr::Options`.

3. https://github.com/testdouble/standard